

Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №1

Вычислительная математика

Вариант 8

Метод Гаусса с выбором максимального элемента по столбцам

Выполнил: Караганов Павел

Группа: Р3210

Проверил: Рыбаков Степан Дмитриевич

г. Санкт-Петербург,

2026

Цель работы

Изучить и реализовать прямой метод решения системы линейных алгебраических уравнений методом Гаусса с выбором главного элемента по столбцам.

В рамках работы необходимо:

- реализовать решение СЛАУ,
- получить верхнетреугольную матрицу,
- вычислить определитель по треугольной матрице,
- вывести вектор неизвестных,
- вычислить и вывести вектор невязок.

Описание метода и расчётные формулы

Рассматривается система линейных уравнений

$$Ax = b,$$

где $A \in \mathbb{R}^{n \times n}$ — матрица коэффициентов, $x \in \mathbb{R}^n$ — вектор неизвестных, $b \in \mathbb{R}^n$ — вектор правых частей.

В программе реализован метод Гаусса с выбором главного элемента по столбцу. Алгоритм состоит из прямого и обратного хода.

1) Прямой ход (приведение к верхнетреугольному виду)

На шаге k ($k = 0, 1, \dots, n - 1$) выбирается главный элемент в k -м столбце:

$$|a_{pk}| = \max_{i=k..n-1} |a_{ik}|.$$

Если $|a_{pk}| < \varepsilon$, где в программе $\varepsilon = 10^{-12}$, матрица считается вырожденной.

Далее строка p меняется местами со строкой k . Для всех строк $i = k + 1, \dots, n - 1$ вычисляется множитель

$$m_{ik} = \frac{a_{ik}}{a_{kk}},$$

и выполняется исключение переменной:

$$a_{ij}^* := a_{ij} - m_{ik}a_{kj}, \quad j = k + 1..n - 1,$$

$$b_i^* := b_i - m_{ik}b_k,$$

$$a_{ik}^* := 0.$$

После завершения прямого хода получается верхнетреугольная матрица U и преобразованный столбец b' .

2) Обратный ход

Неизвестные находятся снизу вверх:

$$x_i = \frac{b'_i - \sum_{j=i+1..n-1} a'_{ij}x_j}{a'_{ii}}, \quad i = n - 1..0.$$

Если на этом этапе $|a'_{ii}| < \varepsilon$, деление невозможно, поэтому решение останавливается с сообщением об ошибке.

3) Определитель матрицы

Так как после прямого хода получена верхнетреугольная матрица, определитель вычисляется как произведение диагональных элементов с учётом количества перестановок строк k :

$$\det(A) = (-1)^k \prod_{i=0..n-1} a'_{ii}.$$

4) Вектор невязок

Для проверки качества решения вычисляется невязка исходной системы:

$$r = b - Ax,$$

$$r_i = b_i - \sum_{j=0..n-1} a_{ij}x_j, \quad i = 0, \dots, n-1.$$

Чем ближе все компоненты r_i к нулю, тем точнее найдено решение.

Листинг программы

```
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>

using namespace std;

constexpr double EPS = 1e-12;

struct LinearSystem {
    vector<vector<double>> a;
    vector<double> b;
};

struct SolutionResult {
    vector<double> x;
    double determinant;
    vector<vector<double>> triangular_a;
    vector<double> triangular_b;
    vector<double> residual;

    void print_all() const {
        cout << "\nТреугольная матрица с преобразованным столбцом B:\n";
        int n = triangular_a.size();
        cout << fixed << setprecision(4);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cout << setw(10) << triangular_a[i][j] << ' ';
            }
            cout << "| " << setw(10) << triangular_b[i] << "\n";
        }

        cout << "\nРешение системы:\n";
        cout << fixed << setprecision(10);
        for (int i = 0; i < x.size(); i++) {
            cout << "x" << (i + 1) << " = " << x[i] << "\n";
        }
    }
};
```

```

        cout << "\nОпределитель матрицы: " << scientific << setprecision(10) <<
determinant << "\n";

        cout << "\nВектор невязок (r = b - A*x):\n";
        cout << fixed << setprecision(10);
        for (int i = 0; i < residual.size(); i++) {
            cout << "r" << (i + 1) << " = " << residual[i] << "\n";
        }
    }
};

void print_menu(bool has_system) {
    cout << "\n===== ЛР №1. Вариант 8 =====\n";
    cout << "Метод: Гаусса с выбором главного элемента по столбцам\n";
    cout << "Текущая система: " << (has_system ? "задана" : "не задана") << "\n";
    cout << "1 - Ввести систему с клавиатуры\n";
    cout << "2 - Загрузить систему из CSV\n";
    cout << "3 - Показать текущую систему\n";
    cout << "4 - Решить текущую систему\n";
    cout << "0 - Выход\n";
}

int read_int(const string& prompt) {
    cout << prompt;
    string raw;
    if (!getline(cin, raw)) {
        throw runtime_error("ошибка чтения ввода");
    }

    istringstream iss(raw);
    int value;
    char extra;
    if (!(iss >> value) || (iss >> extra)) {
        throw runtime_error("ожидалось целое число");
    }

    return value;
}

LinearSystem read_system_from_keyboard() {
    int n = read_int("Введите размер n (для системы n×n): ");
    if (n <= 0) {
        throw runtime_error("n должно быть положительным");
    }

    vector<vector<double>> a;
    vector<double> b;
    a.reserve(n);
    b.reserve(n);

    cout << "Введите " << n << " строк(и): в каждой " << n
        << " коэффициентов и свободный член (всего " << (n + 1) << " чисел).\n";

    for (int i = 0; i < n; i++) {
        cout << "Строка " << (i + 1) << ": ";
        string line;
        if (!getline(cin, line)) {
            throw runtime_error("ошибка чтения строки");
        }
    }
}

```

```

    }

    istream<double> iss(line);
    vector<double> row;
    row.reserve(n);

    for (int j = 0; j < n; j++) {
        double value;
        if (!(iss >> value)) {
            throw runtime_error("в строке " + to_string(i + 1) + " должно быть " +
                                to_string(n + 1) + " чисел");
        }
        row.push_back(value);
    }

    double r_side;
    if (!(iss >> r_side)) {
        throw runtime_error("в строке " + to_string(i + 1) + " должно быть " +
                            to_string(n + 1) + " чисел");
    }

    double extra;
    if (iss >> extra) {
        throw runtime_error("в строке " + to_string(i + 1) + " должно быть " +
                            to_string(n + 1) + " чисел");
    }

    a.push_back(row);
    b.push_back(r_side);
}

return {a, b};
}

string trim_copy(const string& value) {
    int start = 0;
    while (start < value.size() && isspace(value[start])) {
        start++;
    }

    int end = value.size();
    while (end > start && isspace(value[end - 1])) {
        end--;
    }

    return value.substr(start, end - start);
}

LinearSystem read_system_from_csv_file(const string& path) {
    ifstream file(path);
    if (!file.is_open()) {
        throw runtime_error("не удалось открыть CSV-файл");
    }

    vector<vector<double>> rows;
    string line;
    while (getline(file, line)) {
        string normalized = trim_copy(line);
        if (normalized.empty()) {

```

```

        continue;
    }

    for (char& ch : normalized) {
        if (ch == ';' || ch == ',') {
            ch = ' ';
        }
    }

    stringstream iss(normalized);
    vector<double> values;
    double number;
    while (iss >> number) {
        values.push_back(number);
    }

    if (!iss.eof()) {
        throw runtime_error("файл содержит некорректные числовые значения");
    }

    if (values.size() < 2) {
        throw runtime_error("в каждой строке файла должно быть минимум 2 числа");
    }

    rows.push_back(values);
}

if (rows.empty()) {
    throw runtime_error("CSV-файл пуст");
}

int columns = rows[0].size();
for (int i = 1; i < rows.size(); i++) {
    if (rows[i].size() != columns) {
        throw runtime_error("все строки CSV должны иметь одинаковое число столбцов");
    }
}

int n = columns - 1;
if (n <= 0) {
    throw runtime_error("некорректный формат CSV");
}

if (rows.size() != n) {
    throw runtime_error("для системы nхn в CSV должно быть n строк и n+1 столбец");
}

vector<vector<double>> a;
vector<double> b;
a.reserve(n);
b.reserve(n);

for (int i = 0; i < n; i++) {
    vector<double> row;
    row.reserve(n);
    for (int j = 0; j < n; j++) {
        row.push_back(rows[i][j]);
    }
    a.push_back(row);
}

```

```

        b.push_back(rows[i][n]);
    }

    return {a, b};
}

SolutionResult solve_gauss_column_pivot(const LinearSystem& system) {
    int n = system.a.size();
    vector<vector<double>> a = system.a;
    vector<double> b = system.b;
    int swaps = 0;

    for (int k = 0; k < n; k++) {
        int pivot_row = k;
        double max_value = fabs(a[k][k]);

        for (int i = k + 1; i < n; i++) {
            double value = fabs(a[i][k]);
            if (value > max_value) {
                max_value = value;
                pivot_row = i;
            }
        }

        if (max_value < EPS) {
            throw runtime_error("матрица вырождена или система имеет неединственное
решение");
        }

        if (pivot_row != k) {
            swap(a[k], a[pivot_row]);
            swap(b[k], b[pivot_row]);
            swaps++;
        }

        for (int i = k + 1; i < n; i++) {
            double factor = a[i][k] / a[k][k];
            a[i][k] = 0.0;
            for (int j = k + 1; j < n; ++j) {
                a[i][j] -= factor * a[k][j];
            }
            b[i] -= factor * b[k];
        }
    }

    vector<double> x(n, 0.0);
    for (int i = n - 1; i >= 0; i--) {
        double total = b[i];
        for (int j = i + 1; j < n; j++) {
            total -= a[i][j] * x[j];
        }

        if (fabs(a[i][i]) < EPS) {
            throw runtime_error("нулевой диагональный элемент при обратном ходе");
        }
        x[i] = total / a[i][i];
    }

    double det = 1.0;

```

```

    for (int i = 0; i < n; i++) {
        det *= a[i][i];
    }

    if (swaps % 2 == 1) {
        det = -det;
    }

    vector<double> residual(n);
    for (int i = 0; i < n; i++) {
        double sum = 0.0;
        for (int j = 0; j < n; j++) {
            sum += system.a[i][j] * x[j];
        }
        residual[i] = system.b[i] - sum;
    }

    SolutionResult result;
    result.x = x;
    result.determinant = det;
    result.triangular_a = a;
    result.triangular_b = b;
    result.residual = residual;
    return result;
}

void print_system(const LinearSystem& system) {
    cout << "\nТекущая система:\n";
    int n = system.a.size();
    cout << fixed << setprecision(4);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(10) << system.a[i][j] << ' ';
        }
        cout << "| " << setw(10) << system.b[i] << "\n";
    }
}

int main() {
    bool has_system = false;
    LinearSystem current;

    while (true) {
        print_menu(has_system);

        int choice;
        try {
            choice = read_int("Выберите пункт: ");
        } catch (const exception& err) {
            if (cin.eof() || !cin) {
                break;
            }
            cout << "Ошибка ввода: " << err.what() << "\n";
            continue;
        }

        if (choice == 1) {
            try {
                current = read_system_from_keyboard();
            }

```



```

        has_system = true;
        cout << "Матрица успешно загружена с клавиатуры.\n";
    } catch (const exception& err) {
        cout << "Ошибка: " << err.what() << "\n";
    }
} else if (choice == 2) {
    cout << "Введите путь к CSV-файлу: ";
    string path;
    if (!getline(cin, path)) {
        cout << "Ошибка: не удалось прочитать путь к файлу.\n";
        continue;
    }
    try {
        current = read_system_from_csv_file(path);
        has_system = true;
        cout << "Матрица успешно загружена из CSV.\n";
    } catch (const exception& err) {
        cout << "Ошибка: " << err.what() << "\n";
    }
} else if (choice == 3) {
    if (!has_system) {
        cout << "Система не задана. Сначала введите её (пункт 1 или 2).\n";
        continue;
    }
    print_system(current);
} else if (choice == 4) {
    if (!has_system) {
        cout << "Система не задана. Сначала введите её (пункт 1 или 2).\n";
        continue;
    }
    try {
        SolutionResult result = solve_gauss_column_pivot(current);
        result.print_all();
    } catch (const exception& err) {
        cout << "Ошибка решения: " << err.what() << "\n";
    }
} else if (choice == 0) {
    cout << "Завершение программы.\n";
    break;
} else {
    cout << "Неизвестный пункт меню.\n";
}
}
}

```

Примеры и результаты работы программы

Для тестовой системы

$$\begin{cases} 2x_1 + 1x_2 - 1x_3 = 8 \\ -3x_1 - 1x_2 + 2x_3 = -11 \\ -2x_1 + 1x_2 + 2x_3 = -3 \end{cases}$$

получено решение:

- $x_1 = 2$,
- $x_2 = 3$,
- $x_3 = -1$.

Определитель матрицы: $\det(A) = -1$.

Вектор невязок близок к нулевому:

$$r \approx (0, 0, 0)^T.$$

NB: При запуске программы пользователь вводит матрицу с клавиатуры, после чего в пункте решения выводятся:

- верхнетреугольная матрица с преобразованным столбцом B ,
- вектор неизвестных,
- определитель,
- вектор невязок.

Выводы

В работе реализован метод Гаусса с выбором главного элемента по столбцам для решения СЛАУ. Программа корректно формирует треугольную матрицу, выполняет обратный ход, вычисляет определитель и вектор невязок.

Полученные результаты на тестовых данных подтверждают корректность реализации: найденное решение удовлетворяет исходной системе, а невязка близка к нулю.